Group C – Project Plan

Angel Perez, Justin Emerick, Daniel Nethala, Mathew Goncalves

Saint Leo University

COM 430: Software Engineering

Professor Paquette

May 25, 2026

Introduction

DoIT is a collaborative project management platform designed to support team communication, workflow organization, and project tracking in real time. The application focuses on project activities such as task assignment, progress tracking, and messaging within a single system. In many team environments, communication and coordination are often spread across multiple tools, which can lead to missed deadlines, unclear responsibilities, and reduced productivity. DoIT addresses this issue by providing a workspace where users can efficiently manage projects from start to finish.

As part of its planned enhancements, DoIT will also include an AI-powered assistant designed to support users by suggesting task priorities, summarizing updates, and helping organize workflows more efficiently. This feature is intended to improve productivity and reduce the manual effort required in managing team-based projects. This project follows a DevOps-based development model, progressing through Development, Testing, Staging, and Production environments to ensure structured deployment and system reliability.

Project Plan

The DoIT project will follow a structured DevOps pipeline consisting of Development, Testing, Staging, and Production environments.

Development Phase

During the development phase, the core features of the application will be designed and implemented. Developers will create components such as user authentication, project dashboards, task management systems, and collaboration tools. Code will be maintained through version control systems to support team and code management. An AI assistant will also be developed during this phase to provide task prioritization suggestions, workflow recommendations, and project update summaries based on user activity within the platform.

Testing Phase

After development, the application will move into the testing environment. In this phase, functionality, performance, and security will be evaluated to identify system issues and ensure application reliability. Unit testing and integration testing will be performed to validate that all features operate correctly. The AI assistant will also be tested to verify the accuracy and reliability of generated recommendations and summaries.

Staging Phase

The staging environment will serve as a replica used for final validation and user testing. This phase ensures that the application performs correctly in a realistic environment before public release.

Production Phase

Once testing and staging are successfully completed, the application will be ready to use. Monitoring tools and maintenance procedures will be used to track application performance and overall reliability.

Project Scope

The scope of the DoIT includes the design and development of a collaboration system that supports team productivity, communication, and task management.

In Scope

- User account and authentication system
- Task, assignment, and tracking features
- Team collaboration tools such as messaging

- File sharing within project groups

- Dashboard displaying project progress and activity

- Roles

- AI-powered assistant for task suggestions, workflow support, and summarization

Out of Scope

- Advanced AI automation beyond basic assistance features

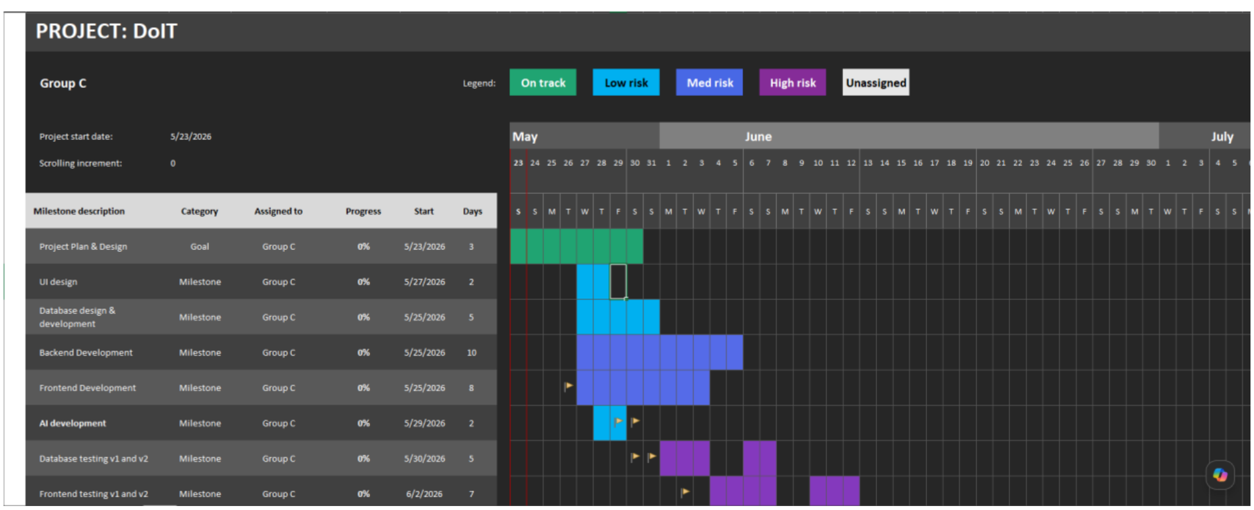
- Mobile application development during the current phase

Schedule (Gantt Chart)

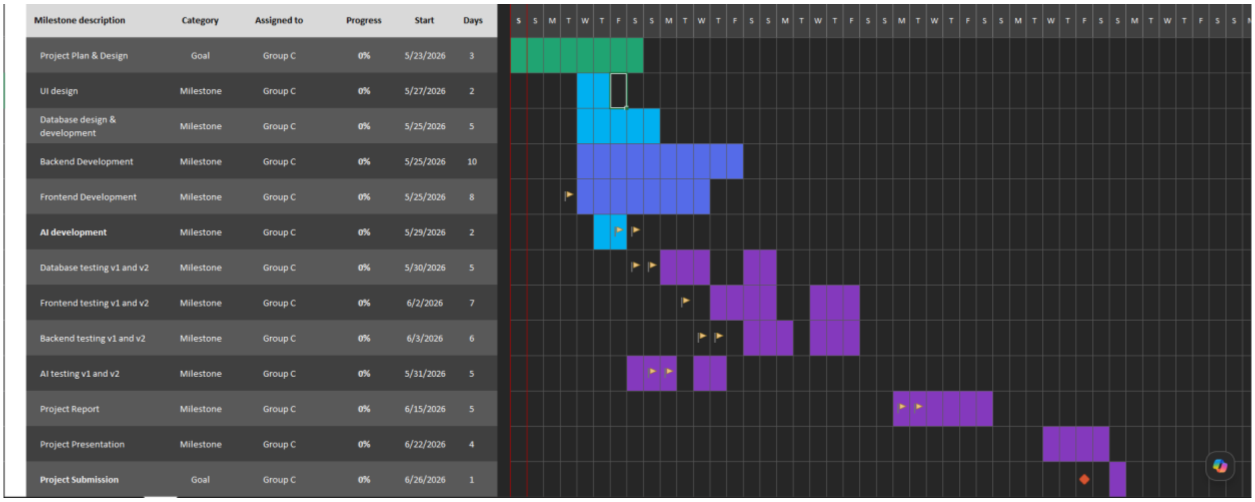
The following Gantt chart outlines the projected development schedule for the DoIT application throughout the software engineering lifecycle. The schedule includes major project milestones, development phases, testing activities, deployment preparation, and final project deliverables used to organize the overall project timeline.

Figure 1

DoIT Project Schedule and Gantt Chart



Milestone description	Category	Assigned to	Progress	Start	Days
Project Plan & Design	Goal	Group C	0%	5/23/2026	3
UI design	Milestone	Group C	0%	5/27/2026	2
Database design & development	Milestone	Group C	0%	5/25/2026	5
Backend Development	Milestone	Group C	0%	5/25/2026	10
Frontend Development	Milestone	Group C	0%	5/25/2026	8
AI development	Milestone	Group C	0%	5/29/2026	2
Database testing v1 and v2	Milestone	Group C	0%	5/30/2026	5
Frontend testing v1 and v2	Milestone	Group C	0%	6/2/2026	7



Milestone description	Category	Assigned to	Progress	Start	Days
Project Plan & Design	Goal	Group C	0%	5/23/2026	3
UI design	Milestone	Group C	0%	5/27/2026	2
Database design & development	Milestone	Group C	0%	5/25/2026	5
Backend Development	Milestone	Group C	0%	5/25/2026	10
Frontend Development	Milestone	Group C	0%	5/25/2026	8
AI development	Milestone	Group C	0%	5/29/2026	2
Database testing v1 and v2	Milestone	Group C	0%	5/30/2026	5
Frontend testing v1 and v2	Milestone	Group C	0%	6/2/2026	7
Backend testing v1 and v2	Milestone	Group C	0%	6/3/2026	6
AI testing v1 and v2	Milestone	Group C	0%	5/31/2026	5
Project Report	Milestone	Group C	0%	6/15/2026	5
Project Presentation	Milestone	Group C	0%	6/22/2026	4
Project Submission	Goal	Group C	0%	6/26/2026	1

Note. The Gantt chart illustrates the projected development schedule for the DoIT application, including project planning, UI design, backend development, testing phases, deployment preparation, and final project delivery milestones.

DevOps Lifecycle / Workflow

The DoIT project will follow a DevOps-inspired workflow to organize how the application moves through development, testing, staging, and production. This process helps the team manage changes, test features before release, reduce errors, and maintain a structured development lifecycle. For this project, the DevOps workflow will be used conceptually to show how the application would move from initial development to a final user-ready environment.

DevOps Environment Overview

The DoIT project will utilize multiple development environments throughout the software engineering lifecycle to support coding, testing, validation, and deployment activities.

Table 1

DevOps Environment Overview for the DoIT Application Lifecycle

Environment	Purpose	Main Activities	Expected Outcome
Development	Initial environment where features are created and updated.	Coding, UI design, GitHub commits, feature updates.	Working features ready for testing.
Testing	Environment used to check functionality and identify issues.	Bug testing, validation, troubleshooting, reviewing requirements.	Defects identified and corrected.
Staging	Final review environment that mirrors production.	Final walkthroughs, deployment checks, user acceptance review.	Application approved for production.
Production	Final environment where the completed application is released.	Final deployment, user access, monitoring, maintenance planning.	User-ready version of DoIT.

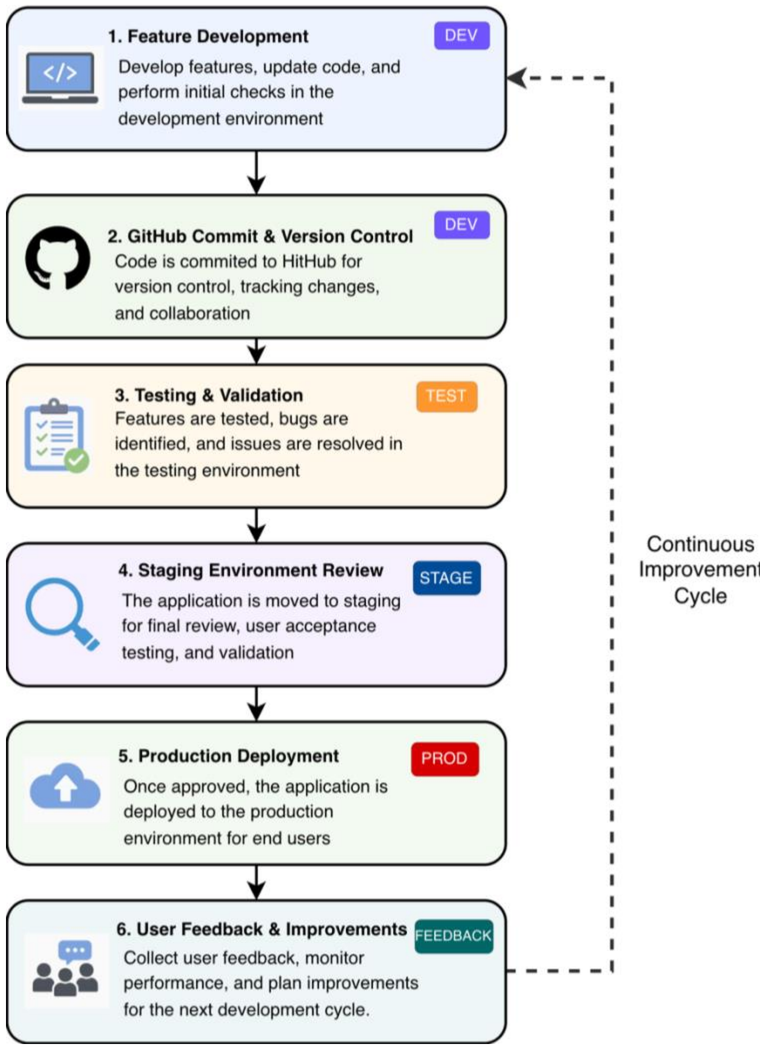
Note. The table summarizes the primary development environments used throughout the DoIT DevOps lifecycle, including development, testing, staging, and production phases.

DevOps Workflow Diagram

The following workflow diagram illustrates how the DoIT application moves through the DevOps lifecycle from development to production deployment

Figure 2

DevOps Workflow Process for the DoIT Application Lifecycle



Note. The diagram illustrates the conceptual DevOps workflow used throughout the DoIT application, including development, testing, staging, production deployment, and continuous improvement processes.

Each environment within the workflow supports a different stage of development, testing, validation, and deployment throughout the software engineering lifecycle.

Workflow and Collaboration Process

Throughout the development lifecycle, the DoIT team will use a collaborative workflow designed to support organization, communication, and continuous improvement. GitHub will be used for version control, file sharing, and tracking project updates throughout development. Microsoft Teams and shared documentation platforms such as Word and Google Docs will support communication, task coordination, and milestone tracking among team members. As new features are developed, they will first move through the development environment before being reviewed and tested for functionality and stability.

Once features and updates have been validated within the testing environment, the application will move into the staging environment for final review and walkthroughs. The staging environment allows the team to simulate a production-ready version of the application before final deployment. After successful validation and approval, the finalized version of DoIT would move into the production environment where the application becomes available to end users. This workflow helps reduce deployment issues, improve organization, and support a more structured software engineering process throughout the project lifecycle.

Continuous Improvement Process

The DevOps lifecycle used for the DoIT project supports continuous improvement throughout the software development process. Feedback collected during testing, staging, and user review phases allows the development team to identify issues, improve functionality, and refine application features over time. As new improvements are identified, updates can be reintroduced into the development environment where additional testing and validation can occur before deployment. This iterative workflow supports Agile-inspired development practices by encouraging ongoing collaboration, communication, and incremental improvement throughout the SDLC process. By continuously evaluating application performance and user feedback, the team can maintain a more organized and adaptable development workflow throughout the project lifecycle. This continuous improvement cycle is illustrated within the DevOps workflow diagram through the feedback loop connecting the production environment back to the development environment.

Data Dictionary

User

- User_ID: Unique identifier for each user
- Username: Name used for login and display
- Email: User email address used for authentication and communication
- Password: user password
- Role: Defines access level (Admin, Member)

Project

- Project_ID: Unique identifier for each project
- Project_Name: Title of the project
- Description: Brief overview of the project
- Start_Date: Project start date
- End_Date: Expected or completion date

Task

- Task_ID: Unique identifier for each task
- Task_Name: Name of the task
- Status: Current state (Pending, In Progress, Completed)
- Due_Date: Deadline for task completion

- Assigned_To: User responsible for the task

Message

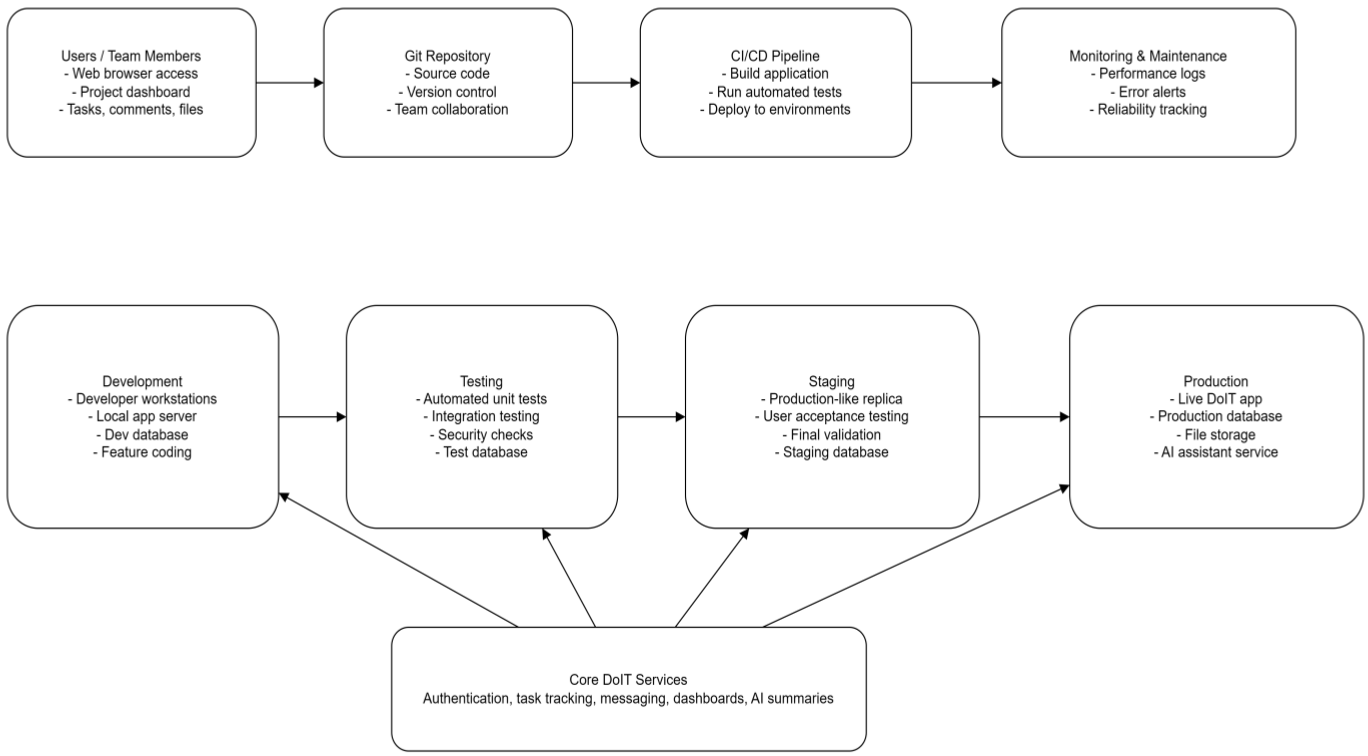
- Message_ID: identifier for messages
- Sender_ID: User sending the message
- Receiver_ID / Group_ID: recipient or team
- Content: Message text
- Timestamp: Time message was sent

Application Diagrams

The following application diagrams provide a visual representation of the primary system interactions and functionality within the DoIT application. These diagrams help illustrate how users interact with the system, how application components are organized, and how core features support communication, collaboration, task management, and project tracking throughout the software engineering lifecycle.

Figure 3

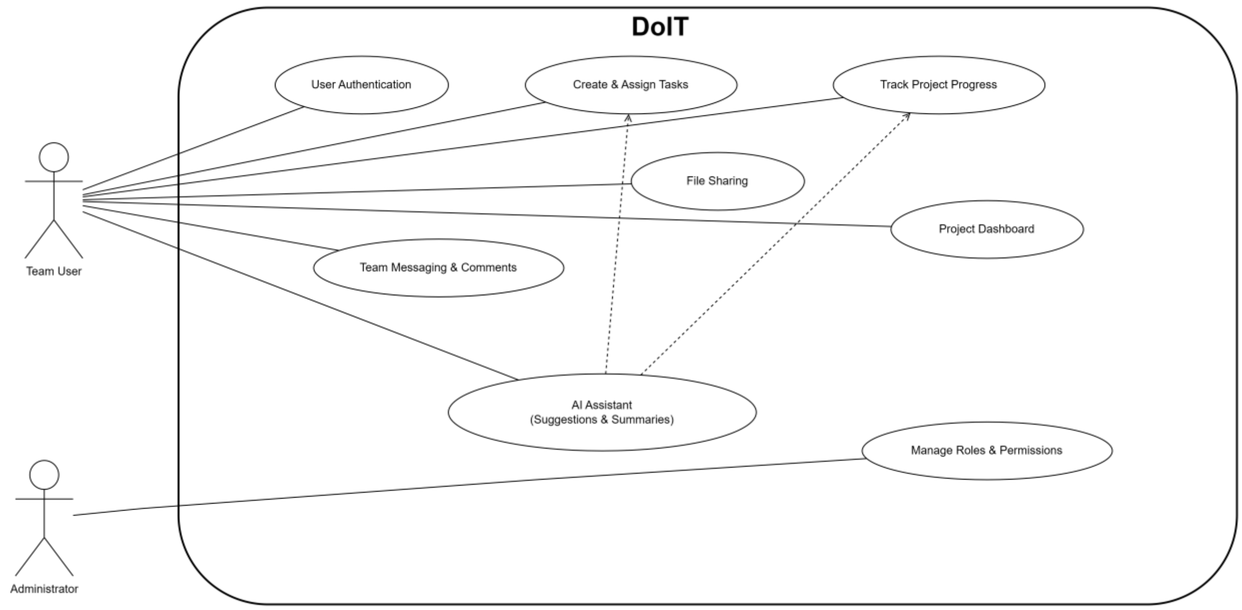
Infrastructure Diagram for the DoIT DevOps Environment



Note. The infrastructure diagram illustrates the conceptual DevOps environment used for the DoIT application, including development, testing, staging, and production environments, along with system workflow and deployment progression.

Figure 4

Use Case Diagram for the DoIT Application



Note. The use case diagram illustrates the primary interactions between team users and the DoIT application, including user authentication, task creation, project tracking, messaging, file sharing, dashboards, and AI-assisted collaboration features.

Risk Assessment

Risk assessment plays an important role throughout the software development lifecycle because technical, operational, scheduling, and security-related issues may impact project success. For the DoIT project, the development team evaluated several potential risks associated with planning, development, testing, deployment, and collaboration throughout the DevOps lifecycle. Identifying these risks early allows the team to implement mitigation strategies that support project stability, improve workflow organization, and reduce the likelihood of deployment or development issues throughout the SDLC process.

Risk Assessment Overview

Table 2

Risk Assessment Overview for the DoIT Application Lifecycle

Risk	Potential Impact	Mitigation Strategy
Scope creep	Delayed development and increased project complexity	Prioritize core features and control feature expansion
Uneven team participation	Missed deadlines and workflow bottlenecks	Maintain communication and track team responsibilities
Software bugs and integration	Reduced functionality and deployment instability	Perform incremental testing and validation throughout development
Timeline constraints	Limited testing and incomplete features	Follow milestone scheduling and prioritize critical functionality
Deployment/ environment issues	Problems transitioning between Dev, Test, Stage, and Production environments	Use staged validation and environment review processes
Security and privacy concerns	Unauthorized access or exposure of project data	Implement secure authentication and role-based access controls

Note. The table summarizes potential project, technical, operational, and security-related risks identified throughout the DoIT software development lifecycle, along with their potential impacts and mitigation strategies used to support project stability and workflow organization.

Each identified risk has the potential to impact the overall stability, organization, or development timeline of the DoIT application if not properly managed throughout the software engineering lifecycle.

Project and Technical Risks

One of the primary project risks associated with the DoIT application involves scope creep throughout the software development lifecycle. As development progresses, additional ideas, features, and enhancements may increase overall project complexity beyond the original scope and timeline expectations. Expanding functionality too quickly could create workflow bottlenecks, delay testing phases, and increase the likelihood of deployment instability. To reduce this risk, the development team will prioritize core application functionality and follow a structured development approach aligned with project milestones and SDLC planning objectives.

Additional technical risks may occur throughout development, testing, and deployment phases of the DevOps lifecycle. Since the DoIT platform contains multiple interconnected components such as task management, collaboration tools, notifications, and user authentication systems, software bugs or integration conflicts may impact overall application stability. Differences between development, testing, staging, and production environments may also create deployment inconsistencies if changes are not properly validated before release. To minimize these risks, the development team will utilize version control practices, staged testing procedures, incremental validation, and collaborative review processes throughout the project lifecycle.

Security and Privacy Risks

Since the DoIT platform stores user information, project data, task assignments, and collaborative communications, security and privacy concerns must also be considered throughout the software development lifecycle. Weak authentication controls, improper user permissions, or insecure handling of application data could potentially expose sensitive project information or allow unauthorized access to system features. In addition, file-sharing and collaboration features may introduce additional security risks if user activity and permissions are not properly managed within the application environment.

To reduce security and privacy risks, the development team will incorporate basic security-focused development practices throughout the DevOps lifecycle. Secure authentication procedures, role-based access controls, controlled user permissions, and staged testing environments will help improve application stability and reduce the likelihood of unauthorized access or accidental exposure of project data. Continuous testing and validation throughout development and staging phases will also support the identification and correction of potential security-related issues before production deployment.

Conclusion

DoIT provides collaborative designed to support team organization, communication, and project management. As a result, these features will ensure that teams can use various tools for collaborative projects. Also, DoIT will help teams meet deadlines and goals for projects. This project plan will ensure that DoIT will be completed within a limited amount of time. This plan has determined the scope of the overall project which helps prevent this project from going beyond scope and encourages this application to be developed on time. Several constraints and risks were assessed to uncover any uncertainties or complexities with this project. The Gantt Chart provides proper scheduling of different stages within the development of the application. The infrastructure and application diagrams help identify the different dev, test, stage, and production environments for this project. Furthermore, this plan will ensure that DoIT is a fully functional application to be presented.